18. Introduction to Paging

Paging

Concept of Paging

- Paging **splits up** address space into **fixed-sized** unit called a **page**.
  Process Address Space를 Fixed-Size의 Page라는 단위로 나눈다.
    - 기존의 Segmentation: variable size of logical segments(code, stack, heap, etc.)
      Segmentation에선 Segment라는 가변 사이즈의 Logical Unit으로 나누었던 것과 대조적!

        - segmentation과 paging이 다르다!!
- With paging, **physical memory** is also **split** into some number of pages called a **page frame**.
    - 물리적인 메모리는 **Page Frame**으로 나눔: page frame 0, page frame 1 ...
    - 가상 메모리는 **Page**로 나눔: page 0, page 1 ...
        - 둘의 크기는 같음
- **Page table** per process is needed **to translate** the virtual address to physical address.
  가상 주소를 물리 주소로 변환하기 위해 **각 프로세스마다 페이지 테이블이 필요하다**

Advantages of Paging

Flexibility (유연성)

- Supporting the abstraction of address space effectively
  Address Space에 대한 Abstraction, 즉, Memory Virtualization을 효과적으로 보조한다.
- Don’t need assumption how heap and stack grow and are used
  heap과 stack이 어떻게 자라고 어떻게 쓰이는지 가정이 필요 없음

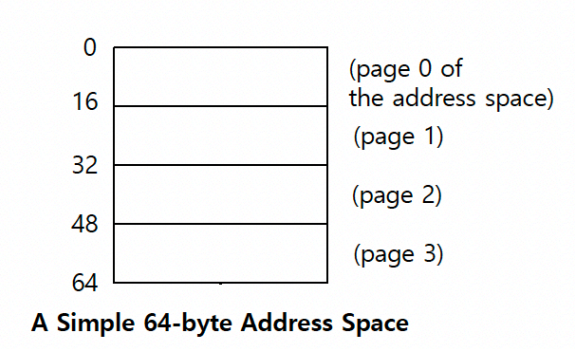
Simplicity (간편성)

- ease of free-space management
- The page in address space and the page frame are the **same size.**
- Easy to allocate and keep a free list

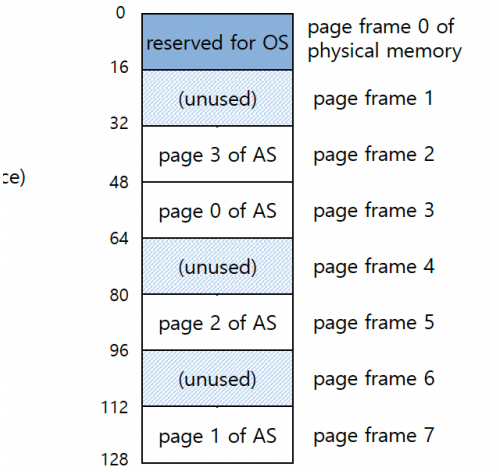
Example: A Simple Paging

- 우측: 128-byte physical memory with 16 bytes page frames
    - 128 byte(2^7) / 16(2^4) byte = 8 → 8개의 page frames
    - 7 bits로 구성 ⇒ ㅁㅁㅁ/ㅁㅁㅁㅁ
    - 0번은 OS가 쓴다고 가정

- 좌측: 64-byte address space with 16 bytes pages
  - 6 bits로 구성 ⇛ □ □/□ □ □ □
  - 4개의 pages



A Simple 64-byte Address Space



64-Byte Address Space Placed In Physical Memory

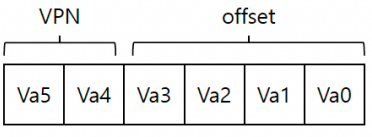
- mapping 관계: **page table** → 이건 어디에 있어야 하냐? memory에 있어야함

page number	page frame number
0	3
1	7
2	5
3	2

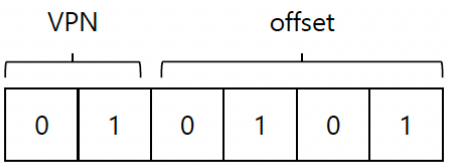
Address Translation

Two components in the virtual address

- **VPN**: virtual page number
- **Offset**: offset within the page



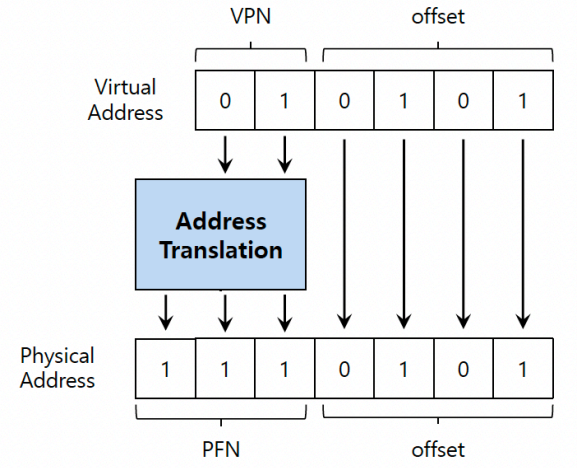
Example: virtual address 21 in 64-byte address space → VPN : 1, Offset: 0101



> **64-byte address space** = 주소 공간의 **크기**가 64바이트
>
> **64-bit address space** = 주소를 표현하는 **비트 수**가 64개
>
> - Byte는 **공간의 크기** (메모리가 얼마나 큰지)
> - Bit는 **주소 표현 비트 수** (주소값을 몇 비트로 쓰는지)

Example: Address Translation

- The virtual address 21 in 64-byte address space



Page Tables

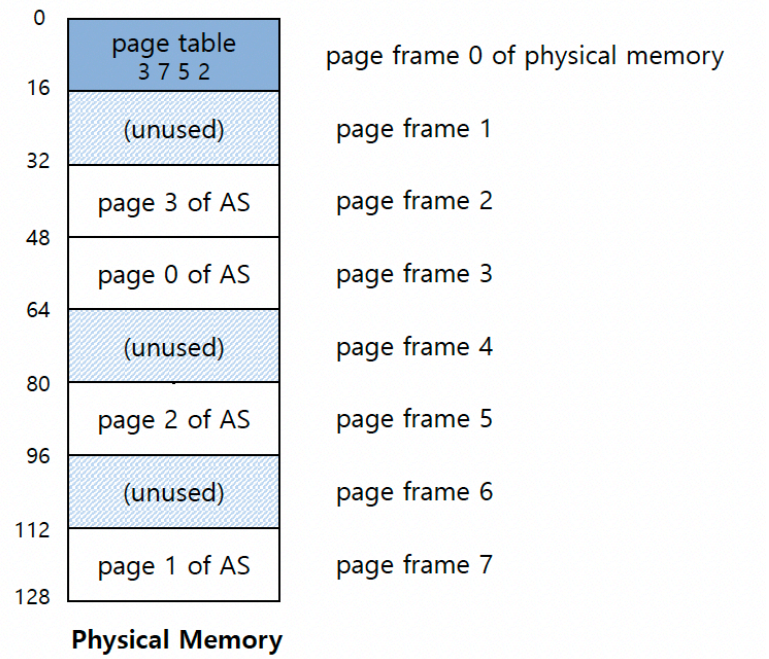
> 2^10 = K, 2^20 = M, 2^30 = G

Where Are Page Tables Stored?

- Page tables can get awfully large
  - 32-bit address space, 그리고 4KB pages 가진 컴퓨터

        - 4K = 2^12임 ⇒ 하위 12 bits 가 offset, 상위 20 bits가 VPN
            - page는 2^20개가 있을 것
                - 2^32 / 2^12 = 2^20 = 1 million(entry가 1 million개
            - 각 entry가 4 byte라고 한다면 (32bit니까)
                - page table의 크기: $4MB$ = 2^20 $entries$ * 4 $Bytes$(32bit 니까) $per\ page\ table\ entry$
- Page tables for each process are stored in memory
    - process가 1개면 4MB
    - 10개면 40MB
    - ...
    - 250개면 1GB ⇒ 너무 크잖슴~
        - 결국에는 physical memory에 두어야 함

Example: Page Table in Kernel Physical Memory



What Is In The Page Table?

- The page table is a **data structure** that is used to map the virtual address to physical address.
    - Simplest form: a linear page table, an array
- The OS **indexes** the array by VPN, and looks up the page-table entry.

- segmentation의 경우 MMU가 계산해줬었음
- paging의 경우 MMU에 넣을 수가 없어 → paging table을 linear하게 물리적 메모리에 넣어야함
    - 매번 memory access가 발생함

Common Flags Of Page Table Entry

Valid Bit

Indicating whether the particular translation is valid
Address Translation이 유효한지를 나타냄

Protection Bit

Indicating whether the page could be read from, written to, or executed from
Page가 Read/Write될 수 있는지, 또는 Execution될 수 있는지

Present Bit

Indicating whether this page is in physical memory or on disk(swapped out)
Page가 Physical Memory에 있는지, 아니면 Disk에 있는지를 나타냄
없으면 page fault 발생?

Dirty Bit

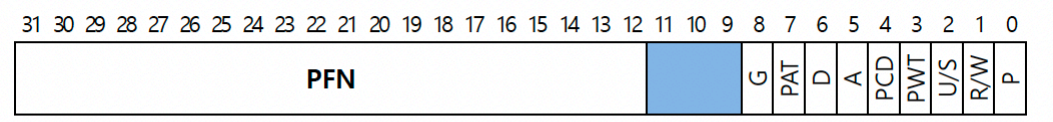
Indicating whether the page has been modified since it was brought into memory
Page가 Memory에 올라온 이후에 수정된 적이 있는지

> 수정된 Page이면 최대한 Disk로 다시 돌려놓지 않고, 'Dirty하지 않은(수정되지 않은)' Page들은 Memory에서 우선적으로 축출한다. 왜? 효율적이기 때문!

Reference Bit(Accessed Bit)

Indicating that a page has been accessed

Example: x86 Page Table Entry



An x86 Page Table Entry(PTE)

- P: present
- R / W: read / write bit
- U / S: supervisor
- A: accessed bit
- D: dirty bit
- PFN: the page frame number

Paging: Too Slow

- To find a location of the desired PTE, the **starting location** of the page table is **needed**
  page table의 시작 주소를 알아야 한다!
- For every memory reference, paging requires the OS to perform one **extra memory reference**.

Accessing Memory With Paging

```
// Extract the VPN from the virtual address
VPN = (VirtualAddress & VPN_MASK) >> SHIFT

// Form the address of the page-table entry (PTE)
PTEAddr = PTBR + (VPN * sizeof(PTE))
// PTBR은 page table base register, 페이지 테이블의 시작 주소
```

```
// Fetch the PTE
PTE = AccessMemory(PTEAddr)

// Check if process can access the page
if(PTE.Valid == False)
        RaiseException(SEGMENTATION_FAULT)

else if (CanAccess(PTE.ProtectBits) == False)
        RaiseException(PROTECTION_FAULT)

else
        // Access is OK: form physical address and fetch it
        offset = VirtualAddress & OFFSET_MASK
        PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
        Register = AccessMemory(PhysAddr)
```

A Memory Trace

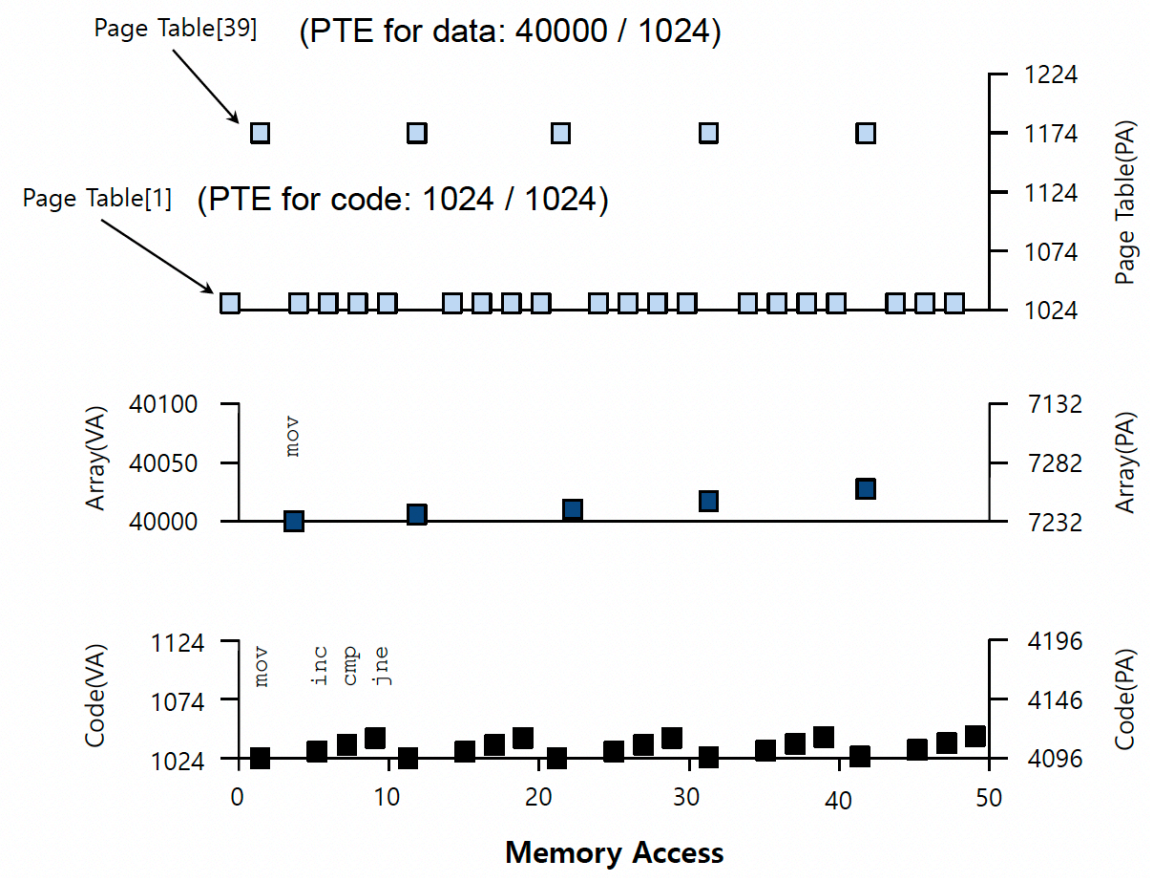
```
int array[1000];
...
for (i = 0; i < 1000; i++)
        array[i] = 0;
```

```
prompt> gcc -o array array.c -Wall -o
prompt>./array
```

Memory access

```
0x1024 movl $0x0,(%edi,%eax,4) //[edi+eax*4]= 0
0x1028 incl %eax
0x102c cmpl $0x03e8,%eax //0000 0011 1110 1000₂ = 1000₁₀
0x1030 jne 0x1024
```

%edi는 array 주소



Code(VA): 1024, 1028, 102c, 1030

총 5번의 메모리 접근을 해야한다?(instruction fetch도 포함?)

왜 40000 / 1024일까

L1이 instruction cache (명령어 저장)

TLB는 MMU 안에? VA로 저장?